

Trade Study of Pipelined 32-bit Ripple Carry Adder

Authors:
WhisperingRock

September 11, 2026

Chapter 1

Timing Analysis

Variables used to explain timing:

- n : the number of full adders. Also the number of bits added.
- t_{dFA} : the worst case carry propagation delay through one FullAdder module.
- t_{reg} : delay for register CLK-to-Q and setup.
- t_{skew} : clock skew and timing margin.
- k : the number of pipeline stages.

1.1 RippleCarryAdder

The combination logic `RippleCarryAdder` doesn't have a clock signal so we must only consider the propagation variable t_{dFA} .

Since there are 8 bits propagated in sequence, each RCA has a summation delay of

$$t_{dRCA} = 8 \cdot t_{dFA}$$

The **latency** of the RCA is our sequential propagation.

$$L_{RCA} = t_{dRCA}$$

The **throughput** of the RCA is

$$\text{Throughput}_{RCA} = \frac{1}{L_{RCA}}$$

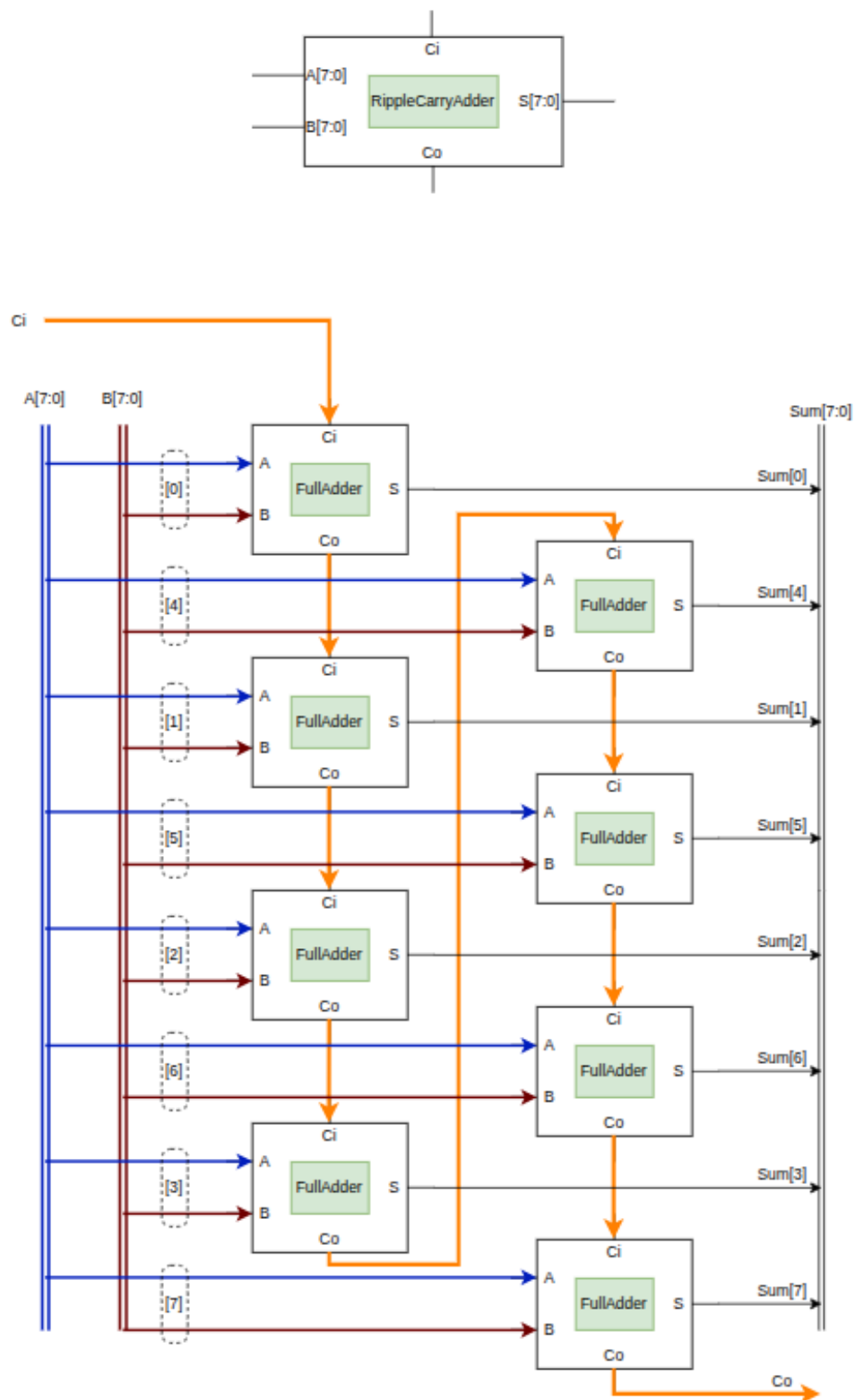


Figure 1.1: Block Diagram of the 8-bit RippleCarryAdder

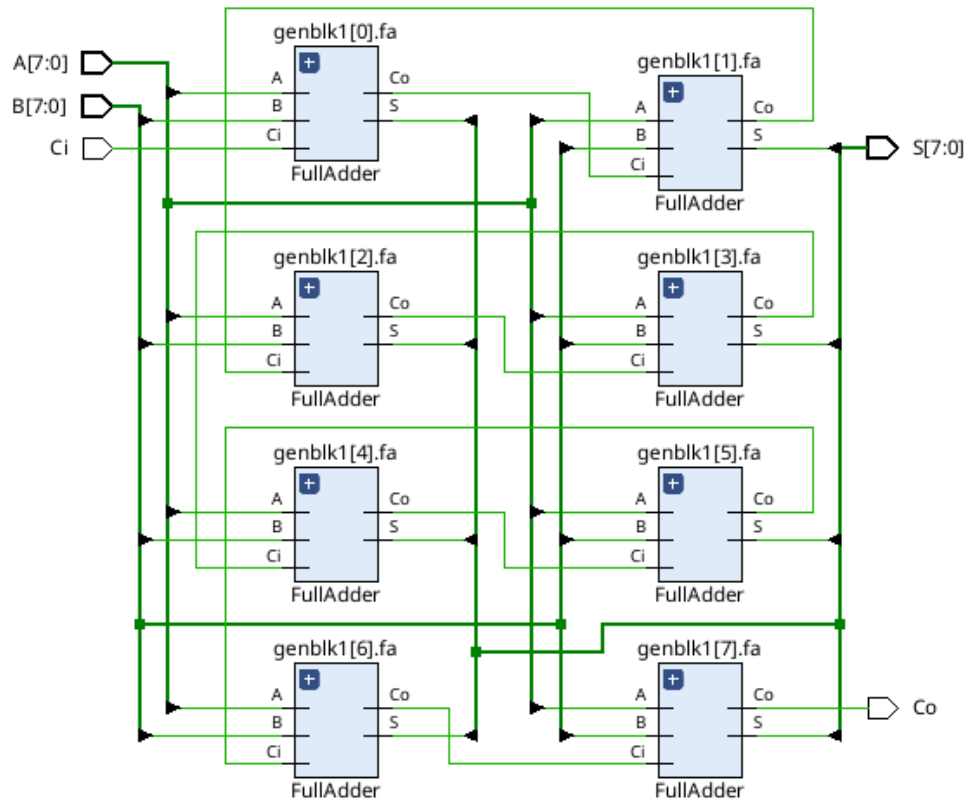


Figure 1.2: RTL of the 8-bit RippleCarryAdder

Code for RCA and test are [2.3](#) and [2.4](#). Code for the FullAdder and test are [2.1](#) and [2.2](#).

1.2 WordAdder

The **WordAdder** is four RCAs in series due to the carry bit. The propagation here is

$$t_{dWA} = 4 \cdot t_{dRCA} = 32 \cdot t_{dFA}$$

The **latency** of the WordAdder is our sequential propagation.

$$L_{WA} = t_{dWA} = 32 \cdot t_{dFA}$$

The **throughput** of the WA is

$$\text{Throughput}_{WA} = \frac{1}{L_{WA}} = \frac{1}{32 \cdot t_{dFA}}$$

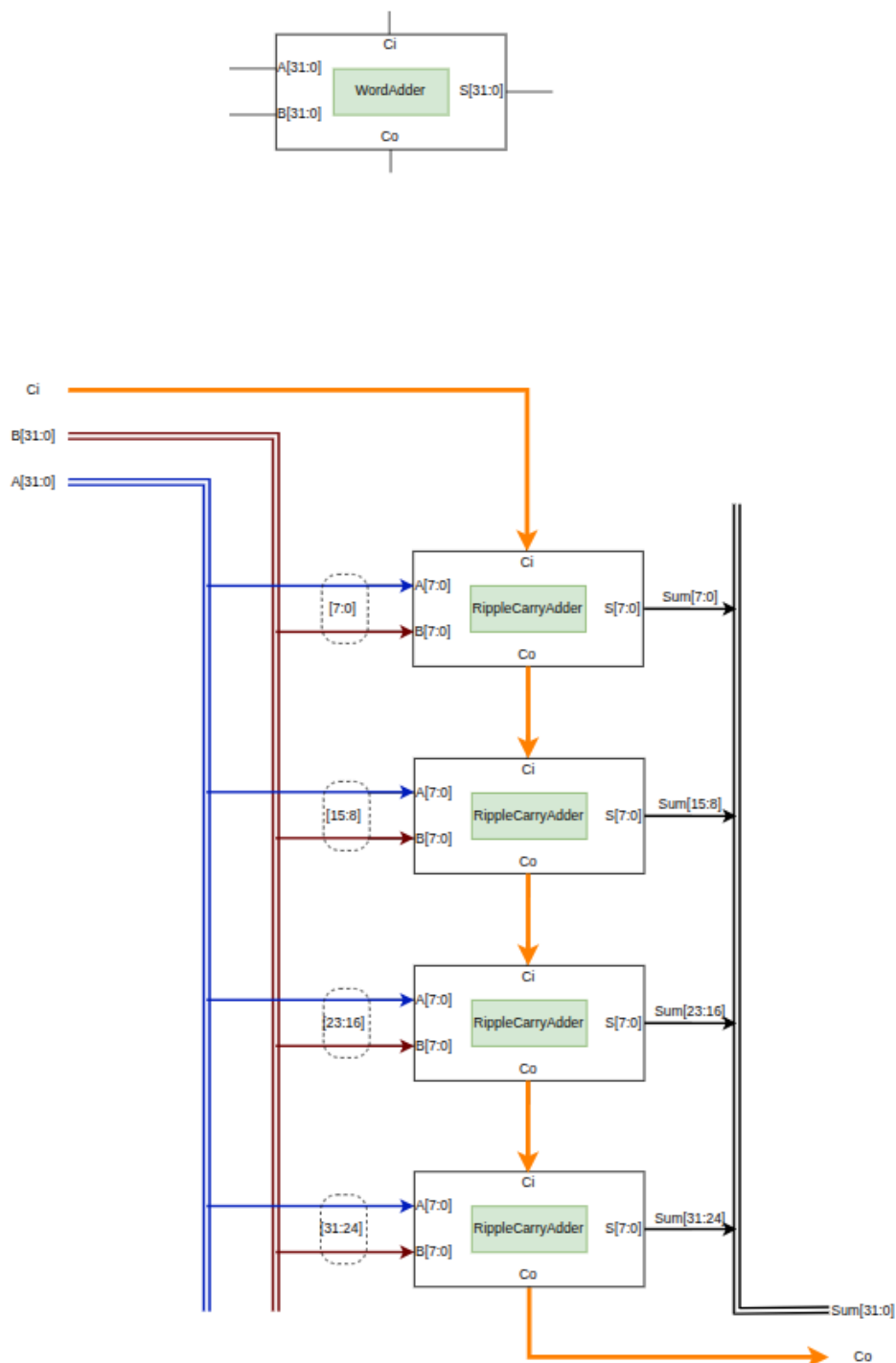


Figure 1.3: Block Diagram of the 32-bit WordAdder

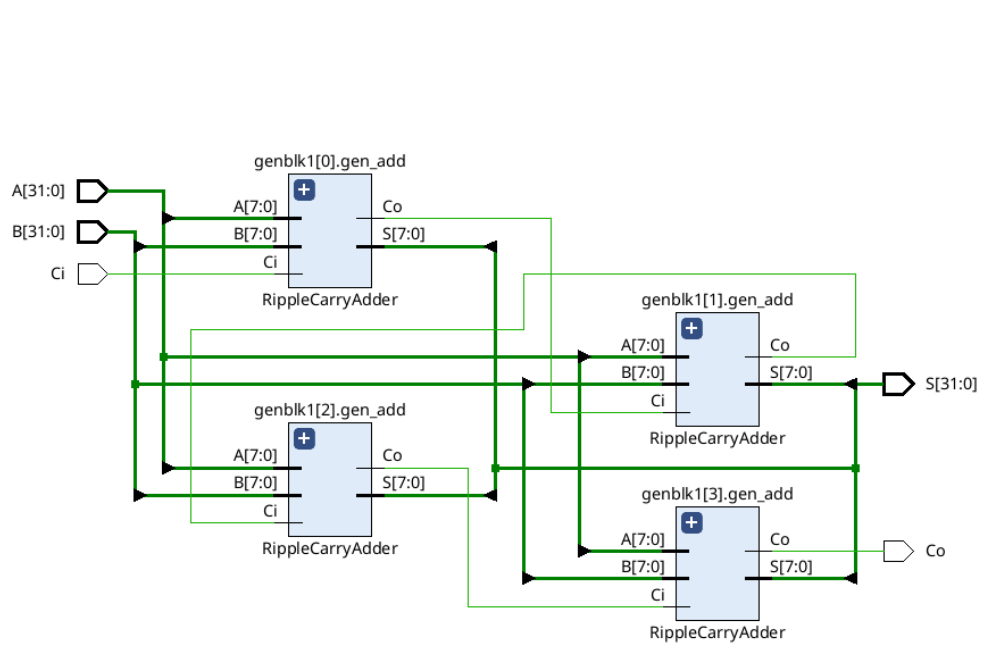


Figure 1.4: RTL of the 32-bit WordAdder

Code for WA and test are [2.5](#) and [2.6](#).

1.3 PipedWordAdder

The piped word adder requires fine-grain calculations. Consider:

- Calculating delays in stages s.t. a single stage must sequentially propagate some bits through the FullAdders. Here, for our 32-bit word, each of the four stages must have time to propagate 8-bits through.

$$t_{stage} \approx \left\lceil \frac{n}{k} \right\rceil t_{dFA} = 8 \cdot t_{dFA}$$

- The minimal clock must be greater than the stage delay plus the delays required for clock skew and register storage.

$$T_{CLK,p} \geq t_{stage} + t_{reg} + t_{skew}$$

Therefore,

the **latency** for adding two words together takes four clock cycles, always.

$$L_{PWA} = k \cdot T_{CLK,p}$$

however, the **throughput** depends on the vacancy of the pipeline. If the pipeline is emptied before processing another pair of words, we must wait 4 cycles for an output. If the pipeline was initially full, then 4 results will be output on each clock cycle. Consider:

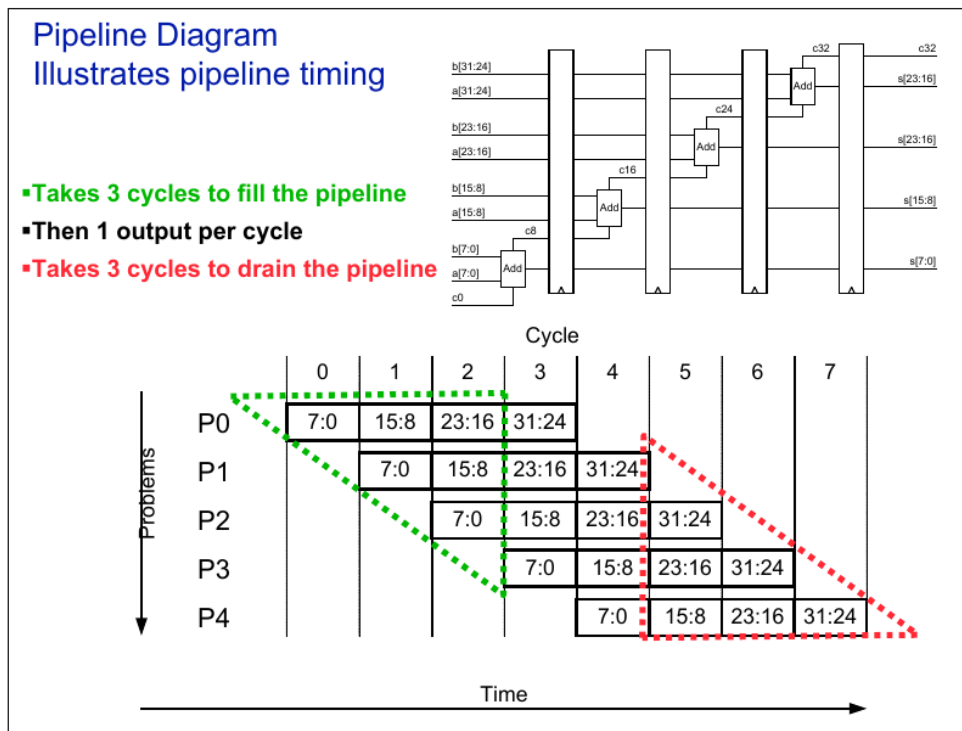


Figure 1.5: Filling and Emptying the Pipe

$$\text{Throughput}_{PWA} = \begin{cases} \frac{1}{4 \cdot T_{CLK,p}} & , \text{ if pipe registers are empty} \\ \vdots & , \vdots \\ \frac{1}{T_{CLK,p}} & , \text{ if pipe registers are full} \end{cases}$$

Ideally we have enough work to keep the adder full at all times to utilize the pipeline performance boost.

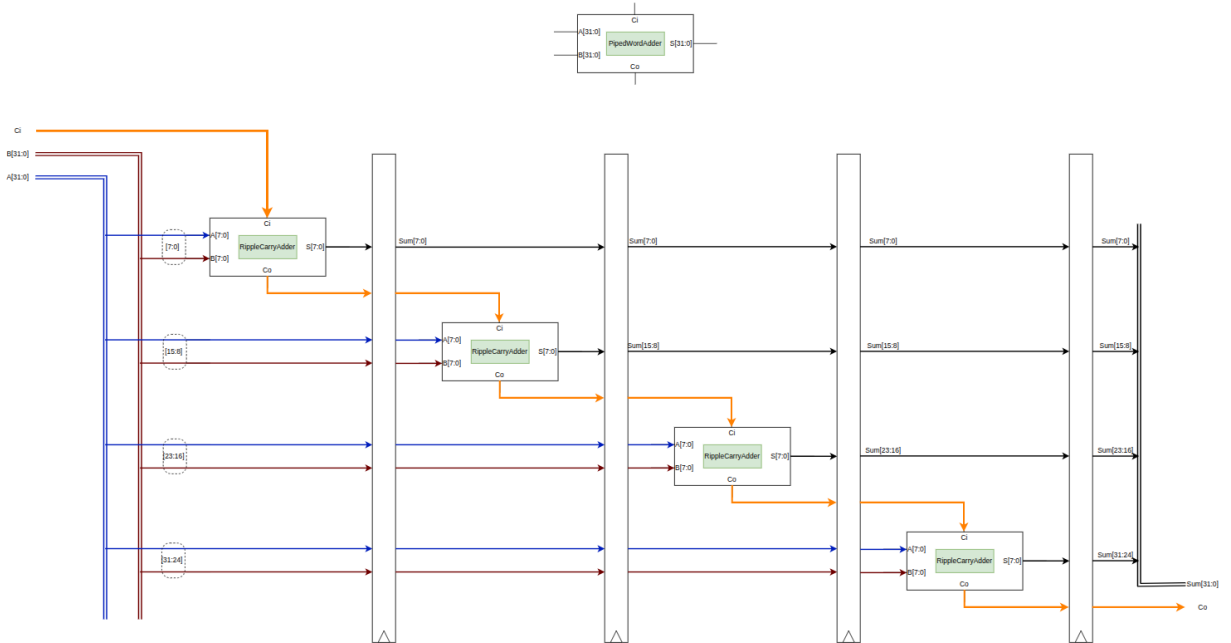


Figure 1.6: Block Diagram of the 32-bit Piped WordAdder

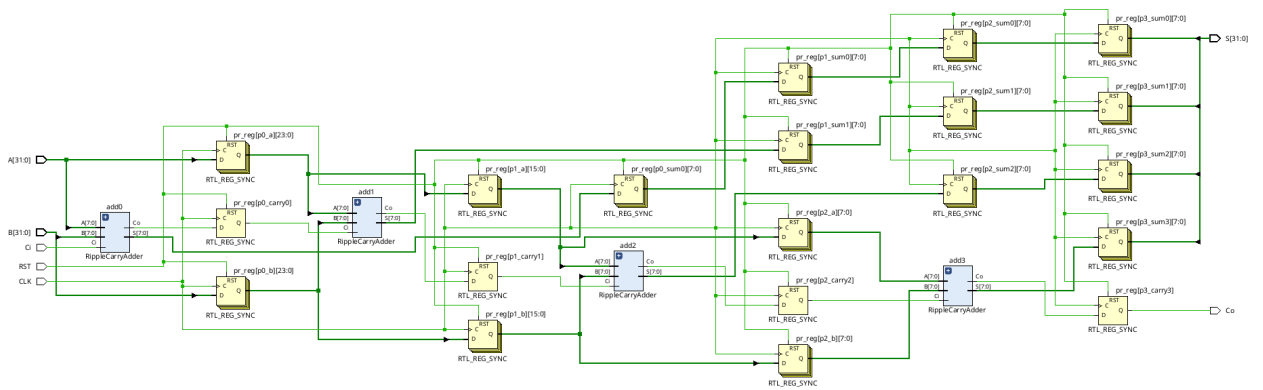


Figure 1.7: RTL of the 32-bit Piped WordAdder

Code for PWA and test are [2.7](#) and [2.8](#).

1.4 Comparison Example

Consider an example where

- $n = 32$ bits
- $k = 4$ stages
- $t_{dFA} = 100ps$
- $t_{reg} = 50ps$
- ignore t_{skew}

	Non-Pipelined	Empty Pipeline	Full Pipeline
Latency(s)	3200p	$\geq 3400p$	$\geq 3400p$
Throughput(ops/s)	312.5 M	294 M	1,176 M

Table 1.1: Calculation of Pipelined vs. Not Pipelined Ripple Word Adder

From our simple example, pipelining increases our latency overhead by adding in more registers but can greatly increase our throughput as long as we can maintain a full pipe.

Throughput performance boost $\approx \frac{1176}{312} \approx 3.77\times$

1.5 Testing on hardware

I'm skeptical this is possible with the 100MHz Basys3 dev board with current timing estimates for the pipeline in the single-digit nanosecond... **todo**

Chapter 2

Appendix

2.1 Design Code

Listing 2.1: FullAdder.sv

```
1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 09/10/2026 10:33:51 AM
7  // Design Name:
8  // Module Name: FullAdder
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module FullAdder(
24     input logic    A, B, Ci,
25     output logic   S, Co
26 );
27
28     assign S      = A ^ B ^ Ci;
29     assign Co     = (A & B) | (A & Ci) | (B & Ci);
30
31     // Zero fun version
32     //assign {Co, S} = A + B + Ci;
33
34 endmodule
```

Listing 2.2: FullAdder_tb.sv

```
1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 09/10/2026 10:42:12 AM
7  // Design Name:
8  // Module Name: FullAdder_tb
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module FullAdder_tb();
24
25     // ~~~~ imports ~~~~
26     import tb_utils_pkg::*;
27
28     // ~~~~ locals ~~~~
29     // test
30     testcase      tc;
31     logic [31:0]  tnum;
32     logic         clk;
33
34     // Input {C, A, B} Output{Sum, Carry}
35     logic [1:0]  adderLUT [0:7];
36
37     // wire
38     logic        a, b, c;
39     logic        sum, carry;
40
```

```

41 // ~~~~ instances ~~~~
42 FullAdder UUT(
43     .A          (a),
44     .B          (b),
45     .Ci         (c),
46     // ~~
47     .S          (sum),
48     .Co         (carry)
49 );
50
51 // ~~~~ heartbeat ~~~~
52 always begin
53     clk <= !clk;
54     #5;
55 end
56
57 // ~~~~ testing ~~~~
58 initial begin
59
60     // ~~ class instances ~~
61     tc
62         = new();
63
64     // ~~ defaults ~~
65     clk
66         = 1'b1;
67     adderLUT
68         =
69         '{
70             // no carry in
71             3'b000: 2'b0_0,
72             3'b001: 2'b1_0,
73             3'b010: 2'b1_0,
74             3'b011: 2'b0_1,
75
76             // carry in
77             3'b100: 2'b1_0,
78             3'b101: 2'b0_1,
79             3'b110: 2'b0_1,
80             3'b111: 2'b1_1,
81
82             default: 2'b0_0
83         };
84
85     // ~~ TC1 ~~
86     tc.new_test("");
87     tnum = tc.get_testnum();
88     for(bit [3:0] i = 4'b0000; i < 4'b1000; i++) begin
89         $display("Ci = %b A = %b B = %b", i[2], i[1], i[0]);
90         c = i[2];
91         a = i[1];
92         b = i[0];
93         #5;
94         assert(carry == adderLUT[i][0]) else tc.err("Carry Out Error");
95         assert(sum == adderLUT[i][1]) else tc.err("Sum Error");
96         #5;
97     end
98     tc.test_done();
99
100 end
101 endmodule

```

Listing 2.3: RippleCarryAdder.sv

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 09/10/2026 11:24:21 AM
7  // Design Name:
8  // Module Name: RippleCarryAdder
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module RippleCarryAdder
24 # (parameter BITWIDTH = 8)
25 (
26     input logic [BITWIDTH-1:0] A, B,
27     input logic Ci,
28     output logic [BITWIDTH-1:0] S,
29     output logic Co
30 );
31
32 // ~~~~ locals ~~~~
33 logic [BITWIDTH:0] carry;
34 genvar i;
35
36 // ~~~~ comb ~~~~
37 assign carry[0] = Ci;
38 assign Co = carry[BITWIDTH];
39
40 // ~~~~ instances ~~~~
41 generate
42     for(i = 0; i < BITWIDTH; i++) begin
43         FullAdder fa(
44             .A          (A[i]),

```

```

45         .B          (B[i]),
46         .Ci         (carry[i]),
47         // ~~
48         .S          (S[i]),
49         .Co         (carry[i+1])
50     );
51     end
52     endgenerate
53
54
55 endmodule

```

Listing 2.4: RippleCarryAdder_tb.sv

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 09/10/2026 11:43:28 AM
7  // Design Name:
8  // Module Name: RippleCarryAdder_tb
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21 class adder_transaction #(int WIDTH = 8);
22
23     // ~~ attributes ~~
24     rand logic [WIDTH-1:0] a;
25     rand logic [WIDTH-1:0] b;
26     rand logic             c;
27
28 endclass
29
30
31
32 module RippleCarryAdder_tb();
33
34     // ~~~~ imports ~~~~
35     import tb_utils_pkg::*;
36
37     // ~~~~ locals ~~~~
38     // const
39     localparam W = 8;
40
41     // test
42     testcase      tc;
43     logic [31:0]  tnum;
44     logic         clk;
45     logic [W:0]   ans;
46     adder_transaction #(W) tr;
47
48
49     // wire
50     logic [W-1:0] a, b;
51     logic         c;
52     logic [W-1:0] sum;
53     logic         carry;
54
55     // ~~~~ instances ~~~~
56     RippleCarryAdder #(W) UUT(
57         .A          (a),
58         .B          (b),
59         .Ci         (c),
60         // ~~
61         .S          (sum),
62         .Co         (carry)
63     );
64
65     // ~~~~ heartbeat ~~~~
66     always begin
67         clk <= !clk;
68         #5;
69     end
70
71     // ~~~~ testing ~~~~
72     initial begin
73
74         // ~~ class instances ~~
75         tc = new();
76
77         tr = new();
78         tr.srandom(32'h1234_ABCD); // seed #
79
80         // ~~ defaults ~~
81         clk = 1'b1;
82
83         // ~~ TC1 ~~
84         tc.new_test("Randomization");
85         tnum = tc.get_testnum();
86         for(int i = 0; i < 10000; i++) begin
87
88             if(!tr.randomize()) begin
89                 $fatal(1, "Randomization Failed");
90             end
91
92             a = tr.a;
93             b = tr.b;

```

```

94         c = tr.c;
95         ans = a + b + c;
96         #5;
97         assert(carry == ans[W]) else tc.err("Carry Out Error");
98         assert(sum == ans[W-1:0]) else tc.err("Sum Error");
99         #5;
100     end
101     tc.test_done();
102
103     tc.new_test("Floor");
104     tnum = tc.get_testnum();
105     a = 0;
106     b = 0;
107     c = 0;
108     #5;
109     assert(carry == 0) else tc.err("Carry Out Error");
110     assert(sum == 0) else tc.err("Sum Error");
111     #5;
112     tc.test_done();
113
114     tc.new_test("Ceililng");
115     tnum = tc.get_testnum();
116     a = 8'hFF;
117     b = 8'hFF;
118     c = 1'b1;
119     #5;
120     assert(carry == 1'b1) else tc.err("Carry Out Error");
121     assert(sum == 8'hFF) else tc.err("Sum Error");
122     #5;
123     tc.test_done();
124
125     tc.new_test("Rollover w/ carry");
126     tnum = tc.get_testnum();
127     a = 8'hFF;
128     b = 8'h00;
129     c = 1'b1;
130     #5;
131     assert(carry == 1'b1) else tc.err("Carry Out Error");
132     assert(sum == 8'h00) else tc.err("Sum Error");
133     #5;
134
135     a = 8'h00;
136     b = 8'hFF;
137     c = 1'b1;
138     #5;
139     assert(carry == 1'b1) else tc.err("Carry Out Error");
140     assert(sum == 8'h00) else tc.err("Sum Error");
141     #5;
142     tc.test_done();
143
144     tc.new_test("Rollover w/o carry");
145     tnum = tc.get_testnum();
146     a = 8'hFF;
147     b = 8'h01;
148     c = 1'b0;
149     #5;
150     assert(carry == 1'b1) else tc.err("Carry Out Error");
151     assert(sum == 8'h00) else tc.err("Sum Error");
152     #5;
153
154     a = 8'h01;
155     b = 8'hFF;
156     c = 1'b0;
157     #5;
158     assert(carry == 1'b1) else tc.err("Carry Out Error");
159     assert(sum == 8'h00) else tc.err("Sum Error");
160     #5;
161     tc.test_done();
162
163     $stop; // stop for timing
164
165 end
166
167
168
169 endmodule

```

Listing 2.5: WordAdder

```

1  'timescale 1ns / 1ps
2  //////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 09/10/2026 12:27:13 PM
7  // Design Name:
8  // Module Name: FourAdd
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 //////////////////////////////////////
21
22
23 module FourAdd(
24     input logic [31:0] A, B,
25     input logic Ci,
26     output logic [31:0] S,
27     output logic Co
28 );

```

```

29
30 // ~~~~ locals ~~~~
31 genvar i;
32 // consts
33 localparam ADDWIDTH = 8;
34 // wires
35 logic [4:0] carry;
36
37
38 // ~~~~ instances ~~~~
39 generate
40     for(i = 0; i < 4; i++) begin
41         RippleCarryAdder #(ADDWIDTH) gen_add
42         (
43             .A (A[i*ADDWIDTH +: ADDWIDTH]),
44             .B (B[i*ADDWIDTH +: ADDWIDTH]),
45             .Ci (carry[i]),
46             // ~~
47             .S (S[i*ADDWIDTH +: ADDWIDTH]),
48             .Co (carry[i+1])
49         );
50     end
51 endgenerate
52
53 // ~~~~ comb logic ~~~~
54 assign carry[0] = Ci;
55 assign Co = carry[4];
56
57
58
59 endmodule

```

Listing 2.6: WordAdder_tb

```

1 'timescale 1ns / 1ps
2 //////////////////////////////////////////////////
3 // Company:
4 // Engineer:
5 //
6 // Create Date: 09/10/2026 12:52:30 PM
7 // Design Name:
8 // Module Name: FourAdd_tb
9 // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 //////////////////////////////////////////////////
21
22 class adder_transaction #(int WIDTH = 8);
23
24     // ~~ attributes ~~
25     rand logic [WIDTH-1:0] a;
26     rand logic [WIDTH-1:0] b;
27     rand logic c;
28
29 endclass
30
31 module FourAdd_tb();
32
33     // ~~~~ imports ~~~~
34     import tb_utils_pkg::*;
35
36     // ~~~~ locals ~~~~
37     // const
38     localparam W = 32;
39     localparam HALFCLK = 5;
40
41     // test
42     testcase tc;
43     logic [31:0] tnum;
44     logic clk;
45     logic [W:0] ans;
46     adder_transaction #(W) tr;
47
48
49     // wire
50     logic [W-1:0] a, b;
51     logic c;
52     logic [W-1:0] sum;
53     logic carry;
54
55     // ~~~~ instances ~~~~
56     FourAdd UUT(
57         .A (a),
58         .B (b),
59         .Ci (c),
60         // ~~
61         .S (sum),
62         .Co (carry)
63     );
64
65     // ~~~~ heartbeat ~~~~
66     always begin
67         #HALFCLK;
68         clk <= !clk;
69     end
70
71     // ~~~~ testing ~~~~
72     initial begin
73

```

```

74 // ~~ class instances ~~
75 tc      = new();
76
77 tr      = new();
78 tr.srandom(32'h1234_ABCD); // seed #
79
80 // ~~ defaults ~~
81 clk     = 1'b1;
82
83 // ~~ TC1 ~~
84 tc.new_test("Randomization");
85 tnum = tc.get_testnum();
86     for(int i = 0; i < 10000; i++) begin
87
88         if(!tr.randomize()) begin
89             $fatal(1, "Randomization Failed");
90         end
91
92         a  = tr.a;
93         b  = tr.b;
94         c  = tr.c;
95         ans = a + b + c;
96         #HALFCLK;
97         assert(carry    === ans[W]) else tc.err("Carry Out Error");
98         assert(sum      === ans[W-1:0]) else tc.err("Sum Error");
99         #HALFCLK;
100     end
101 tc.test_done();
102
103 tc.new_test("Floor");
104 tnum = tc.get_testnum();
105     a  = 0;
106     b  = 0;
107     c  = 0;
108     #HALFCLK;
109     assert(carry    === 0) else tc.err("Carry Out Error");
110     assert(sum      === 0) else tc.err("Sum Error");
111     #HALFCLK;
112 tc.test_done();
113
114 tc.new_test("Ceililng");
115 tnum = tc.get_testnum();
116     a  = 32'hFFFF_FFFF;
117     b  = 32'hFFFF_FFFF;
118     c  = 1'b1;
119     #HALFCLK;
120     assert(carry    === 1'b1) else tc.err("Carry Out Error");
121     assert(sum      === 32'hFFFF_FFFF) else tc.err("Sum Error");
122     #HALFCLK;
123 tc.test_done();
124
125 tc.new_test("Rollover w/ carry");
126 tnum = tc.get_testnum();
127     a  = 32'hFFFF_FFFF;
128     b  = 32'd0;
129     c  = 1'b1;
130     #HALFCLK;
131     assert(carry    === 1'b1) else tc.err("Carry Out Error");
132     assert(sum      === 32'd0) else tc.err("Sum Error");
133     #HALFCLK;
134
135     a  = 32'd0;
136     b  = 32'hFFFF_FFFF;
137     c  = 1'b1;
138     #HALFCLK;
139     assert(carry    === 1'b1) else tc.err("Carry Out Error");
140     assert(sum      === 32'd0) else tc.err("Sum Error");
141     #HALFCLK;
142 tc.test_done();
143
144 tc.new_test("Rollover w/o carry");
145 tnum = tc.get_testnum();
146     a  = 32'hFFFF_FFFF;
147     b  = 32'h0000_0001;
148     c  = 1'b0;
149     #HALFCLK;
150     assert(carry    === 1'b1) else tc.err("Carry Out Error");
151     assert(sum      === 32'd0) else tc.err("Sum Error");
152     #HALFCLK;
153
154     a  = 32'h0000_0001;
155     b  = 32'hFFFF_FFFF;
156     c  = 1'b0;
157     #HALFCLK;
158     assert(carry    === 1'b1) else tc.err("Carry Out Error");
159     assert(sum      === 32'd00) else tc.err("Sum Error");
160     #HALFCLK;
161 tc.test_done();
162
163 $stop; // stop for timing
164
165 end
166
167
168
169 endmodule

```

Listing 2.7: PipedWordAdder

```

1 'timescale 1ns / 1ps
2 //////////////////////////////////////
3 // Company:
4 // Engineer:
5 //
6 // Create Date: 09/10/2026 02:11:52 PM
7 // Design Name:
8 // Module Name: FourAdderPipe

```

```

9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
21 typedef struct packed
22 {
23     // pipe reg 0
24     logic [23:0]    p0_a;
25     logic [23:0]    p0_b;
26     logic [7:0]     p0_sum0;
27     logic           p0_carry0;
28
29     // pipe reg 1
30     logic [15:0]    p1_a;
31     logic [15:0]    p1_b;
32     logic [7:0]     p1_sum0;
33     logic [7:0]     p1_sum1;
34     logic           p1_carry1;
35
36     // pipe reg 2
37     logic [7:0]     p2_a;
38     logic [7:0]     p2_b;
39     logic [7:0]     p2_sum0;
40     logic [7:0]     p2_sum1;
41     logic [7:0]     p2_sum2;
42     logic           p2_carry2;
43
44     // pipe reg 3
45     logic [7:0]     p3_sum0;
46     logic [7:0]     p3_sum1;
47     logic [7:0]     p3_sum2;
48     logic [7:0]     p3_sum3;
49     logic           p3_carry3;
50
51 } pipereg_t;
52
53 module FourAdderPipe(
54     input logic [31:0] A, B,
55     input logic       Ci,
56     input logic       CLK,
57     input logic       RST,
58     output logic [31:0] S,
59     output logic      Co
60 );
61
62 // ~~~~ locals ~~~~
63 // ~~ consts ~~
64 localparam      ADDWIDTH = 8;
65
66 // ~~ wires ~~
67 logic [31:0]    sum;
68 logic [3:0]     carry;
69
70 // ~~ regs ~~
71 pipereg_t pr;
72
73
74 // ~~~~ instances ~~~~
75
76 RippleCarryAdder #(ADDWIDTH) add0
77 (
78     .A            (A[7:0]),
79     .B            (B[7:0]),
80     .Ci           (Ci),
81     // ~~
82     .S            (sum[7:0]),
83     .Co           (carry[0])
84 );
85
86 RippleCarryAdder #(ADDWIDTH) add1
87 (
88     .A            (pr.p0_a[7:0]),
89     .B            (pr.p0_b[7:0]),
90     .Ci           (pr.p0_carry0),
91     // ~~
92     .S            (sum[15:8]),
93     .Co           (carry[1])
94 );
95
96 RippleCarryAdder #(ADDWIDTH) add2
97 (
98     .A            (pr.p1_a[7:0]),
99     .B            (pr.p1_b[7:0]),
100    .Ci           (pr.p1_carry1),
101    // ~~
102    .S            (sum[23:16]),
103    .Co           (carry[2])
104 );
105
106 RippleCarryAdder #(ADDWIDTH) add3
107 (
108     .A            (pr.p2_a),
109     .B            (pr.p2_b),
110     .Ci           (pr.p2_carry2),
111     // ~~
112     .S            (sum[31:24]),
113     .Co           (carry[3])
114 );
115
116 // ~~~~ sync logic ~~~~

```



```

117 always_ff @(posedge CLK) begin
118
119     // ~~ priority 0 : reset ~~
120     if(RST == 1'b1) begin
121         pr <= '0;
122     end
123
124     // ~~ priority 1 : move data ~~
125     else begin
126         // ~~ pipe reg 0 (between adds 0:1) ~~
127         pr.p0_a      <= A[31:8];
128         pr.p0_b      <= B[31:8];
129         pr.p0_sum0    <= sum[7:0];
130         pr.p0_carry0 <= carry[0];
131
132         // ~~ pipe reg 1 (between adds 1:2) ~~
133         pr.p1_a      <= pr.p0_a[23:8];
134         pr.p1_b      <= pr.p0_b[23:8];
135         pr.p1_sum0    <= pr.p0_sum0;
136         pr.p1_sum1    <= sum[15:8];
137         pr.p1_carry1 <= carry[1];
138
139         // ~~ pipe reg 2 (between adds 2:3) ~~
140         pr.p2_a      <= pr.p1_a[15:8];
141         pr.p2_b      <= pr.p1_b[15:8];
142         pr.p2_sum0    <= pr.p1_sum0;
143         pr.p2_sum1    <= pr.p1_sum1;
144         pr.p2_sum2    <= sum[23:16];
145         pr.p2_carry2 <= carry[2];
146
147         // ~~ pipe reg 3 (after add 3) ~~
148         pr.p3_sum0    <= pr.p2_sum0;
149         pr.p3_sum1    <= pr.p2_sum1;
150         pr.p3_sum2    <= pr.p2_sum2;
151         pr.p3_sum3    <= sum[31:24];
152         pr.p3_carry3 <= carry[3];
153     end
154 end
155
156
157
158 // ~~~~ async logic ~~~~
159 assign S      = {pr.p3_sum3, pr.p3_sum2, pr.p3_sum1, pr.p3_sum0};
160 assign Co     = pr.p3_carry3;
161
162
163 endmodule

```

Listing 2.8: PipedWordAdder_tb

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 09/10/2026 12:52:30 PM
7  // Design Name:
8  // Module Name: FourAdd_tb
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22 class adder_transaction #(int WIDTH = 8);
23
24     // ~~ attributes ~~
25     rand logic [WIDTH-1:0] a;
26     rand logic [WIDTH-1:0] b;
27     rand logic              c;
28
29 endclass
30
31 module FourAdderPipe_tb();
32
33     // ~~~~ imports ~~~~
34     import tb_utils_pkg::*;
35
36     // ~~~~ locals ~~~~
37     // const
38     localparam W = 32;
39     localparam HALFCLK = 5;
40
41     // test
42     testcase
43     logic [31:0]      tnum;
44     logic             clk;
45     logic             reset;
46     logic [W:0]       ans;
47     adder_transaction #(W) tr;
48
49
50     // wire
51     logic [W-1:0] a, b;
52     logic         c;
53     logic [W-1:0] sum;
54     logic         carry;
55
56     // ~~~~ instances ~~~~
57     FourAdderPipe UUT(

```

```

58     .A          (a),
59     .B          (b),
60     .Ci         (c),
61     .CLK        (clk),
62     .RST        (reset),
63     // ~~
64     .S          (sum),
65     .Co          (carry)
66 );
67
68 // ~~~~ heartbeat ~~~~
69 always begin
70     #HALFCLK;
71     clk <= !clk;
72 end
73
74 // ~~~~ testing ~~~~
75 initial begin
76
77     // ~~ class instances ~~
78     tc          = new();
79
80     tr          = new();
81     tr.srandom(32'h1234_ABCD); // seed #
82
83     // ~~ defaults ~~
84     clk         = 1'b1;
85     ans         = 32'd0;
86     a           = 32'd0;
87     b           = 32'd0;
88     c           = 1'b0;
89
90     // ~~ UUT init ~~
91     reset       = 1'b1;
92     #100;
93     reset       = 1'b0;
94     #100;
95
96     // ~~ TC1 ~~
97     tc.new_test("Adding : randomization");
98     tnum = tc.get_testnum();
99     for(int i = 0; i < 10000; i++) begin
100
101         if(!tr.randomize()) begin
102             $fatal(1, "Randomization Failed");
103         end
104
105         // write data and wait 3 cycles for it to come out
106         a = tr.a;
107         b = tr.b;
108         c = tr.c;
109         ans = a + b + c;
110         #HALFCLK; #HALFCLK;
111         a = 32'd0;
112         b = 32'd0;
113         c = 1'b0;
114         #HALFCLK; #HALFCLK; // takes 3 cycles to get the info out
115         #HALFCLK; #HALFCLK;
116         #HALFCLK;
117         assert(carry == ans[W]) else tc.err("Carry Out Error");
118         assert(sum == ans[W-1:0]) else tc.err("Sum Error");
119         #HALFCLK;
120     end
121     tc.test_done();
122
123     // ~~ sanitize ~~
124     reset = 1'b1;
125     #HALFCLK; #HALFCLK;
126     reset = 1'b0;
127     #HALFCLK; #HALFCLK;
128
129     tc.new_test("Adding : Floor");
130     tnum = tc.get_testnum();
131     a = 0;
132     b = 0;
133     c = 0;
134     #HALFCLK;
135     #HALFCLK; #HALFCLK;
136     #HALFCLK; #HALFCLK;
137     assert(carry == 0) else tc.err("Carry Out Error");
138     assert(sum == 0) else tc.err("Sum Error");
139     #HALFCLK;
140     tc.test_done();
141
142     tc.new_test("Adding : Ceililng");
143     tnum = tc.get_testnum();
144     a = 32'hFFFF_FFFF;
145     b = 32'hFFFF_FFFF;
146     c = 1'b1;
147     #HALFCLK;
148     assert(carry == 1'b1) else tc.err("Carry Out Error");
149     assert(sum == 32'hFFFF_FFFF) else tc.err("Sum Error");
150     #HALFCLK;
151     tc.test_done();
152
153     tc.new_test("Adding : Rollover w/ carry");
154     tnum = tc.get_testnum();
155     a = 32'hFFFF_FFFF;
156     b = 32'd0;
157     c = 1'b1;
158     #HALFCLK;
159     assert(carry == 1'b1) else tc.err("Carry Out Error");
160     assert(sum == 32'd0) else tc.err("Sum Error");
161     #HALFCLK;
162
163     a = 32'd0;
164     b = 32'hFFFF_FFFF;
165     c = 1'b1;

```

```

166         #HALFCLK;
167         assert(carry      === 1'b1)    else tc.err("Carry Out Error");
168         assert(sum        === 32'd0)    else tc.err("Sum Error");
169         #HALFCLK;
170         tc.test_done();
171
172         tc.new_test("Adding : Rollover w/o carry");
173         tnum = tc.get_testnum();
174         a    = 32'hFFFF_FFFF;
175         b    = 32'h0000_0001;
176         c    = 1'b0;
177         #HALFCLK;
178         assert(carry      === 1'b1)    else tc.err("Carry Out Error");
179         assert(sum        === 32'd0)    else tc.err("Sum Error");
180         #HALFCLK;
181
182         a    = 32'h0000_0001;
183         b    = 32'hFFFF_FFFF;
184         c    = 1'b0;
185         #HALFCLK;
186         assert(carry      === 1'b1)    else tc.err("Carry Out Error");
187         assert(sum        === 32'd00)   else tc.err("Sum Error");
188         #HALFCLK;
189         tc.test_done();
190
191         $stop; // stop for timing
192
193     end
194
195
196
197 endmodule

```

2.2 Utility Code

Listing 2.9: Packaged class entities for testing tb_utils_pkg.sv

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 08/25/2026 06:38:30 PM
7  // Design Name:
8  // Module Name: tb_utils_pkg
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22 package tb_utils_pkg;
23
24     /*
25     Class testcase
26
27     - Purpose : track test number and number of errors
28
29
30     */
31     class testcase;
32         // ~~ attributes ~~
33         int unsigned err_cnt;
34         int unsigned testnum;
35
36         // ~~ constructors ~~
37         function new();
38             this.err_cnt = 0;
39             this.testnum = 0;
40         endfunction
41
42         // ~~ methods ~~
43         function void new_test(input string mes);
44             this.testnum++;
45             this.err_cnt = 0;
46             $display("\nStarting TC%0d : %s", this.testnum, mes);
47         endfunction
48
49         function void err(input string mes);
50             this.err_cnt++;
51             $display("\t\tERROR: %s", mes);
52         endfunction: err
53
54         function void test_done();
55             if(err_cnt > 0) begin
56                 $display("TC%0d complete : %0d ERRORS\n", this.testnum, this.err_cnt);
57             end
58
59             else begin
60                 $display("TC%0d complete : SUCCESS\n", this.testnum);
61             end
62         endfunction
63
64         function int get_testnum();
65             return this.testnum;
66         endfunction
67
68         function void print_subtest(input string mes);
69             $display("|-%s", mes);

```

```
70         endfunction
71     endclass
72 endpackage
```